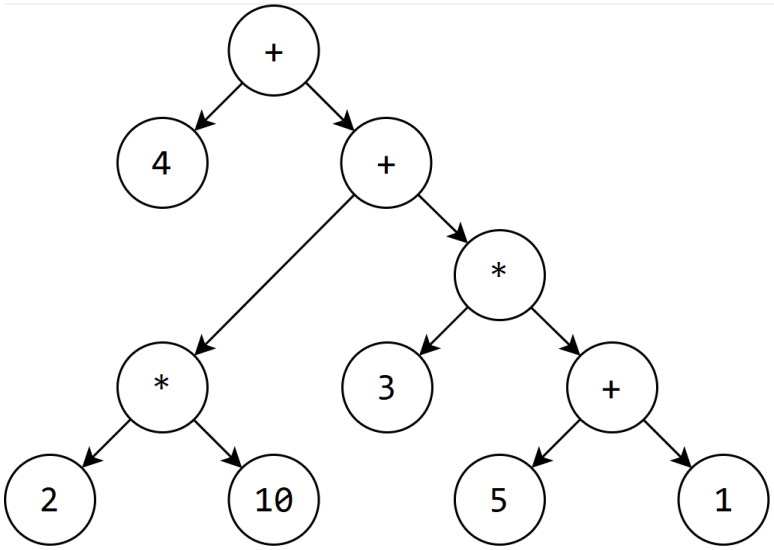
SE311 SPRING 2025-2026 / 11-03-2026
Lab Work 03 – Composite Pattern

An **Abstract Syntax Tree (AST)** is a tree representation of the syntactic structures of source code, capturing the syntax in a hierarchical form. Each node in the **AST** represents a construct in the code, such as expressions, statements, or operators.

In this lab, you will simulate the arithmetic expression evaluation in **AST**. The arithmetic expressions will be limited to only binary operators (+,*,-). / will be handled differently.

Following is an example AST.



A binary arithmetic expression in a programming language contains operator and operands. The operands can be another expression or just a number. As you can understand from the figure above, an expression node denotes an arithmetic expression and operand nodes denote operands. Therefore, you should match expression nodes and operand nodes of the AST with Composite Pattern's participants. (Participant Mapping)

You will have additional subtypes used for differentiating among different operation types. Example: **`SummationNode`** subtype is used for addition.

Implement a polymorphic method called "**eval()**" that will do the following:

It will evaluate its operands and return a value. If the operand is just a number it will return the value of the corresponding number. Otherwise, it will apply the mathematical operation to the two operands and return the result (The type of the node as described above will determine the Mathematical operation to be performed.).

HINT: NO IF STATEMENTS IN **`eval()`** METHOD, IT WILL BE SOMETHING LIKE THIS FOR **`SummationNode`**:

```
public int eval(){
      return firstOperandNode.eval() + secondOperandNode.eval();
}
```

Implement this using the **Transparent** version of the Composite pattern. The list of components will be defined in the Component. You are free to choose which data structure to use in the Composite. Create a **`DivisionNode,`** class however; do not allow this to be added to the AST. Also, do not allow adding components into leaf nodes. When both adding operations are requested for that cases throw an exception at those parts.

BONUS: Write a display method that will display it in prefix notation.
Example: + 4 + * 2 10 * 3 + 1 5